МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра ВТ

ОТЧЕТ
по лабораторной работе № 4
по дисциплине «Распределённые системы»
Тема: Исследование репликации данных и
разрешения конфликтов в распределенной системе

Студенты гр. 2307 _____ Почекунин А. П.
_____ Буянов. М.Г.
_____ Подберёзский А.Д.
Преподаватель _____ Соколова Е.А.

Санкт-Петербург
2026

Цель работы: изучение принципов репликации данных в распределенных системах, а также анализ механизмов возникновения и разрешения конфликтов при параллельных операциях записи с использованием моделей ослабленной согласованности.

Задание:

В ходе выполнения лабораторной работы необходимо разработать модель распределённой системы хранения данных, состоящей из нескольких независимых узлов, и исследовать процессы репликации и синхронизации при параллельном доступе.

Для этого необходимо выполнить следующие задачи:

1. Спроектировать и реализовать программную модель распределённой системы, состоящую как минимум из трёх узлов. Каждый узел должен хранить локальную копию (реплику) одного и того же объекта данных (например, числовое значение, строку или запись с версией).
2. Реализовать механизм асинхронной репликации данных между узлами, предусмотрев возможность имитации задержек передачи сообщений.
3. Смоделировать сценарий параллельных операций записи на разных узлах, приводящий к возникновению конфликта версий данных.
4. Реализовать механизм обнаружения конфликтов на основе сравнения версий данных (например, с использованием временных меток, векторных часов или счётчиков версий).
5. Применить один из алгоритмов разрешения конфликтов (**Last Write Wins**, векторные часы или упрощённый вариант Version Vectors) для приведения реплик к согласованному состоянию.
6. Провести анализ полученных результатов, оценив влияние выбранного алгоритма на согласованность данных и поведение системы в условиях задержек.

Требования к реализации:

- Система должна состоять минимум из трёх узлов, каждый с собственной репликой.

- Обмен данными между узлами должен быть асинхронным; допускается имитация сетевых задержек.
- Для отслеживания версий данных разрешается использовать логические или счётчиковые временные метки.

Реализация может быть выполнена в виде консольного приложения на любом языке программирования высокого уровня (Python, Java, C++ и др.) без использования реальных сетевых соединений — допускается имитация взаимодействия через вызовы функций или очереди сообщений.

Ход выполнения работы:

Для реализации модели распределённой системы с репликацией данных было разработано консольное приложение на языке Python с использованием веб-фреймворка Flask. Каждый узел системы представляет собой отдельный экземпляр Flask-приложения, хранящий локальную копию (реплику) общего объекта данных — в данном случае целочисленного значения и связанной с ним временной метки. Система состоит из трёх узлов, взаимодействующих друг с другом посредством HTTP-запросов.

При запуске узла ему присваивается уникальный идентификатор (node_id) и порт, на котором он будет принимать входящие соединения. Каждый узел хранит актуальное значение local_value и временную метку последнего обновления local_timestamp (в наносекундах). Для обеспечения потокобезопасности при одновременном доступе к этим данным из разных запросов используется блокировка threading.Lock.

Основные функции узла:

- Чтение данных — эндпоинт GET /value возвращает текущее значение узла, его временную метку и идентификатор узла. Это позволяет в любой момент проверить состояние конкретной реплики.

```
PS C:\Study\RS_Labs\Lab4\src> curl http://localhost:5001/value


StatusCode        : 200
StatusDescription : OK
Content           : {"node_id":1,"timestamp":0,"value":0}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 38
                    Content-Type: application/json
                    Date: Mon, 02 Mar 2026 19:24:22 GMT
                    Server: Werkzeug/3.1.6 Python/3.14.3

                    {"node_id":1,"timestamp":0,"value":...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 38], [Content-Type, application/json], [Date, Mon, 02 Mar 2026 19:24:22 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : System.__ComObject
RawContentLength  : 38
```

Рис.1. Возвращённое значение узла, временная метка и его идентификатор

```
127.0.0.1 - - [02/Mar/2026 22:18:00] "GET /value HTTP/1.1" 200 -
```

Рис.2. Сообщение об успешном запросе значения узла

- Запись данных — эндпоинт POST /value принимает новое значение в теле запроса. При получении запроса значение узла обновляется, временная

метка устанавливается равной текущему системному времени (в наносекундах). Сразу после обновления запускается асинхронный процесс рассылки изменений двум другим узлам. Это позволяет не задерживать ответ клиенту и имитирует асинхронную репликацию в реальных распределённых системах.

```
PS C:\Study\RS_Labs\Lab4\src> Invoke-WebRequest  -Uri "http://localhost:5001/value"  -Method POST  -Headers @{ "Content-Type" = "application/json" }  -Body '{"value": 100}'


StatusCode        : 200
StatusDescription : OK
Content           : {"node_id":1,"timestamp":1772483095662838700,"value":100}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 58
                    Content-Type: application/json
                    Date: Mon, 02 Mar 2026 20:24:55 GMT
                    Server: Werkzeug/3.1.6 Python/3.14.3

                    {"node_id":1,"timestamp":1772483095...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 58], [Content-Type, application/json], [Date, Mon, 02 Mar 2026 20:24:55 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : System.__ComObject
RawContentLength  : 58
```

```
PS C:\Study\RS_Labs\Lab4\src> curl http://localhost:5002/value


StatusCode        : 200
StatusDescription : OK
Content           : {"node_id":2,"timestamp":1772483095662838700,"value":100}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 58
                    Content-Type: application/json
                    Date: Mon, 02 Mar 2026 20:25:23 GMT
                    Server: Werkzeug/3.1.6 Python/3.14.3

                    {"node_id":2,"timestamp":1772483095...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 58], [Content-Type, application/json], [Date, Mon, 02 Mar 2026 20:25:23 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : System.__ComObject
RawContentLength  : 58
```

```
PS C:\Study\RS_Labs\Lab4\src> curl http://localhost:5003/value


StatusCode        : 200
StatusDescription : OK
Content           : {"node_id":3,"timestamp":1772483095662838700,"value":100}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 58
                    Content-Type: application/json
                    Date: Mon, 02 Mar 2026 20:25:28 GMT
                    Server: Werkzeug/3.1.6 Python/3.14.3

                    {"node_id":3,"timestamp":1772483095...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 58], [Content-Type, application/json], [Date, Mon, 02 Mar 2026 20:25:28 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : System.__ComObject
RawContentLength  : 58
```

Рис.3. Данные узлов после записи новых данных

```
127.0.0.1 - - [02/Mar/2026 23:24:55] "POST /value HTTP/1.1" 200 -
Node 1: Propagated to node 2
Node 1: Propagated to node 3
```

Рис.4. Сообщение об успешной перезаписи значений в узлах

- Синхронизация — эндпоинт POST /sync предназначен для приёма обновлений от других узлов. При получении запроса узел сравнивает присланную временную метку со своей локальной. Если присланная метка больше, локальное значение и метка заменяются присланными. В случае равенства меток применяется правило разрешения конфликтов на основе идентификатора узла: узел с меньшим node_id считается победителем, и его значение принимается. Если локальная метка новее, обновление игнорируется. Таким образом реализован алгоритм разрешения конфликтов на основе временных меток с дополнительным правилом (tie-breaker), что соответствует стратегии Last Write Wins.

```
PS C:\Study\RS_Labs\Lab4\src> Invoke-WebRequest  -Uri "http://localhost:5001/value"  -Method POST  -Headers @{ "Content-Type" = "application/json" }  -Body '{"value": 200}'


StatusCode        : 200
StatusDescription : OK
Content           : {"node_id":1,"timestamp":1772483148730127800,"value":200}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 58
                    Content-Type: application/json
                    Date: Mon, 02 Mar 2026 20:25:48 GMT
                    Server: Werkzeug/3.1.6 Python/3.14.3

                    {"node_id":1,"timestamp":1772483148...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 58], [Content-Type, application/json], [Date, Mon, 02 Mar 2026 20:25:48 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : System.__ComObject
RawContentLength  : 58


PS C:\Study\RS_Labs\Lab4\src> Invoke-WebRequest  -Uri "http://localhost:5002/value"  -Method POST  -Headers @{ "Content-Type" = "application/json" }  -Body '{"value": 300}'


StatusCode        : 200
StatusDescription : OK
Content           : {"node_id":2,"timestamp":1772483160417688800,"value":300}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 58
                    Content-Type: application/json
                    Date: Mon, 02 Mar 2026 20:26:00 GMT
                    Server: Werkzeug/3.1.6 Python/3.14.3

                    {"node_id":2,"timestamp":1772483160...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 58], [Content-Type, application/json], [Date, Mon, 02 Mar 2026 20:26:00 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : System.__ComObject
RawContentLength  : 58
```

```
PS C:\Study\RS_Labs\Lab4\src> curl http://localhost:5001/value


StatusCode        : 200
StatusDescription : OK
Content           : {"node_id":1,"timestamp":1772483160417688800,"value":300}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 58
                    Content-Type: application/json
                    Date: Mon, 02 Mar 2026 20:26:05 GMT
                    Server: Werkzeug/3.1.6 Python/3.14.3

                    {"node_id":1,"timestamp":1772483160...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 58], [Content-Type, application/json], [Date, Mon, 02 Mar 2026 20:26:05 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : System.__ComObject
RawContentLength  : 58



PS C:\Study\RS_Labs\Lab4\src> curl http://localhost:5002/value


StatusCode        : 200
StatusDescription : OK
Content           : {"node_id":2,"timestamp":1772483160417688800,"value":300}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 58
                    Content-Type: application/json
                    Date: Mon, 02 Mar 2026 20:26:05 GMT
                    Server: Werkzeug/3.1.6 Python/3.14.3

                    {"node_id":2,"timestamp":1772483160...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 58], [Content-Type, application/json], [Date, Mon, 02 Mar 2026 20:26:05 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : System.__ComObject
RawContentLength  : 58
```

```
PS C:\Study\RS_Labs\Lab4\src> curl http://localhost:5003/value


StatusCode        : 200
StatusDescription : OK
Content           : {"node_id":3,"timestamp":1772483160417688800,"value":300}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 58
                    Content-Type: application/json
                    Date: Mon, 02 Mar 2026 20:26:06 GMT
                    Server: Werkzeug/3.1.6 Python/3.14.3

                    {"node_id":3,"timestamp":1772483160...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 58], [Content-Type, application/json], [Date, Mon, 02 Mar 2026 20:26:06 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : System.__ComObject
RawContentLength  : 58
```

Рис.5. Проверка стратегии LWW

```
Node 2: Updating from 100 to 200 (timestamp 1772483148730127800 > 1772483095662838700)
127.0.0.1 - - [02/Mar/2026 23:25:49] "POST /sync HTTP/1.1" 200 -
127.0.0.1 - - [02/Mar/2026 23:26:00] "POST /value HTTP/1.1" 200 -
Node 2: Propagated to node 1
Node 2: Propagated to node 3
127.0.0.1 - - [02/Mar/2026 23:26:05] "GET /value HTTP/1.1" 200 -
```

Рис.6. Логи 2-го узла во время проверки

Асинхронная репликация и имитация задержек:

Для рассылки обновлений используется отдельный поток, что гарантирует

неблокирующую работу основного потока обработки запросов. В функции propagate выполняется перебор всех остальных узлов, и для каждого формируется HTTP-запрос с текущим значением, временной меткой и идентификатором узла-отправителя. Перед отправкой каждого запроса вносится случайная задержка с помощью time.sleep(random.uniform(0.1, 0.5)), что имитирует непредсказуемые сетевые задержки и позволяет наблюдать возникновение конфликтов при параллельной записи.

Логирование и наблюдение за работой:

Для наглядности каждый узел выводит в консоль сообщения о ключевых событиях: успешном обновлении значения, получении обновления от другого узла, игнорировании устаревшего обновления, а также об успешной или неудачной рассылке данных соседям. Это позволяет в реальном времени отслеживать состояние системы и процесс разрешения конфликтов.

Вывод:

В ходе выполнения лабораторной работы были изучены и реализованы механизмы обеспечения отказоустойчивости и безопасности в распределённых системах на примере RPC-сервиса с использованием брокера сообщений. Разработанная система включает три основных компонента: RPC-сервис, клиент с логикой таймаутов и повторов, а также брокер сообщений для асинхронной обработки задач.

Реализованный RPC-сервис продемонстрировал способность обрабатывать запросы идемпотентно благодаря использованию уникальных идентификаторов запросов, что исключает дублирование операций при повторных отправках со стороны клиента. Клиент, в свою очередь, был оснащён механизмом таймаута и повторных попыток, что позволяет компенсировать временные сетевые сбои и повышает надёжность взаимодействия.

Внедрение брокера сообщений по модели публикации-подписки обеспечило асинхронную обработку длительных операций: после создания заказа сервис публикует событие, а независимый worker, подписанный на соответствующий

топик, выполняет фоновую работу. Такой подход позволяет разгрузить основной сервис и повысить общую производительность системы.

Таким образом, в работе были успешно реализованы ключевые паттерны построения отказоустойчивых распределённых приложений: идемпотентность обработки запросов, таймауты и повторные попытки на стороне клиента, а также асинхронная обработка через брокер сообщений. Полученные результаты демонстрируют, что комбинация этих механизмов позволяет создавать системы, устойчивые к сбоям и способные эффективно обрабатывать запросы в условиях нестабильной сети.

Приложение 1. Код программы

```
from flask import Flask, request, jsonify
import threading
import requests
import time
from datetime import datetime
import random
import sys

app = Flask(__name__)

NODE_ID = None
NODES = {
	1: "http://localhost:5001",
	2: "http://localhost:5002",
	3: "http://localhost:5003"
}

local_value = 0
local_timestamp = 0
lock = threading.Lock()

@app.route('/value', methods=['GET'])
def get_value():
	with lock: # locks are used because flask is using multithreading under the hood,
and every endpoint could change local_value/timestamp
		return jsonify({
			'node_id': NODE_ID,
			'value': local_value,
			'timestamp': local_timestamp
		})


@app.route('/value', methods=['POST'])
def update_value():
	global local_value, local_timestamp

	data = request.get_json()
	if not data or 'value' not in data:
		return jsonify({'error': 'Missing value'}), 400

	new_value = data['value']

	with lock:
```

```
        local_value = new_value
        local_timestamp = time.time_ns()

        # start non-blocking sync with other 2 nodes
        update_other_nodes()

        return jsonify({
            'node_id': NODE_ID,
            'value': local_value,
            'timestamp': local_timestamp
        })

# this endpoint is only accessed by another node
@app.route('/sync', methods=['POST'])
def sync():
    global local_value, local_timestamp

    data = request.get_json()
    if not data:
        return jsonify({'error': 'Invalid data'}), 400

    received_value = data['value']
    received_timestamp = data['timestamp']
    received_node_id = data['node_id']

    with lock:
        if received_timestamp > local_timestamp:
            print(f"Node {NODE_ID}: Updating from {local_value} to
{received_value} (timestamp {received_timestamp} > {local_timestamp})")
            local_value = received_value
            local_timestamp = received_timestamp
        elif received_timestamp == local_timestamp and received_node_id <
NODE_ID:
            # Tie-breaker: if same timestamp, lower node ID wins
            print(f"Node {NODE_ID}: Same timestamp, node {received_node_id}
< {NODE_ID}, updating")
            local_value = received_value
        else:
            print(f"Node {NODE_ID}: Ignoring update (local timestamp
{local_timestamp} >= {received_timestamp})")

    return jsonify({'status': 'synced'})

# asynchronios
def update_other_nodes():
```

```
    current_value = local_value
    current_timestamp = local_timestamp

    def propagate():
        for node_id, url in NODES.items():
            if node_id == NODE_ID:
                continue

            try:
                requests.post(f"{url}/sync", json={'value': current_value,
'timestamp': current_timestamp, 'node_id': NODE_ID}, timeout=1)
                print(f"Node {NODE_ID}: Propagated to node {node_id}")
            except:
                print(f"Node {NODE_ID}: Failed to reach node {node_id}")

    thread = threading.Thread(target=propagate)
    thread.daemon = True
    thread.start()


if __name__ == '__main__':
    NODE_ID = int(sys.argv[1])
    port = int(sys.argv[2])
    app.run(host='0.0.0.0', port=port, debug=False)
```